UNIVERSIDAD INTERNACIONAL DEL ECUADOR

Facultad de Ingenierías Digitales y Sociales

Carrera de Ingeniería en Sistemas de Información

Programación Orientada a Objetos

Actividad

Proyecto Integrador Fase 2

Sistema de Gestión de Correos Internos (Go) | Grupo 7

Integrantes:

Camila Isabel Paucar Guaranda

Santiago Alejandro Lucina Mena

Pamela Stefania Toapanta Pila

Repositorio en GitHub:

https://github.com/Slucina04060708/PROYECTO-INTEGRADOR

Profesor:

MARÍA DEL CARMEN SALAZAR TORRES

1 de noviembre de 2026

UIDE
Powered by
Arizona State University®

ÍNDICE

1. OBJETIVO DE LA APLICACIÓN ----- 2
2. ACTORES ----- 3
3. ALCANCE ----- 4
   4.1. Operaciones que Realizará la Aplicación ----- 4
   4.2. Operaciones Adicionales del Sistema ----- 4
   4.3. Exclusiones del Sistema ----- 4
4. REQUISITOS ----- 5
   5.1. Requisitos Funcionales y de Integración ----- 5
   5.2. Requisitos No Funcionales ----- 7
5. DIAGRAMA DE CASOS DE USO ----- 9
6. FICHAS DE CASOS DE USO ----- 10
7. MATRIZ DE TRAZABILIDAD ----- 16
8. CAMBIOS RESPECTO A LA FASE 1 ----- 21
9. DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL ----- 23

1. OBJETIVO DE LA APLICACIÓN

El objetivo del sistema es el de gestionar correos internos entre dos sedes de una organización bajo una arquitectura cliente-servidor desacoplada desarrollada en Go. Esta operará sobre una infraestructura de red interconectada de la organización que ofrecerá lo siguiente:

- Autenticación e inicio de sesión seguro mediante la integración con un microservicio externo en FastAPI.
- Redacción y envío de mensajes de correo interno entre colaboradores con registro de marca de tiempo.
- Consulta estructurada de bandejas de entrada y salida, diferenciando visual y lógicamente los mensajes leídos de los no leídos.
- Lectura de mensajes específicos con actualización automática del estado a *leído*
- Eliminación lógica de registros para mantener la trazabilidad e integridad histórica.
- Procesamiento concurrente de múltiples peticiones simultáneas desde diferentes sedes mediante *goroutines* y canales en Go.

Está dirigida a los empleados y colaboradores (usuarios generales) de las distintas sedes de la organización que necesitan un canal de comunicación interno oficial, seguro, centralizado y ordenado para la toma de decisiones administrativas, evitando la pérdida de información por canales informales.

2. ACTORES

- **Usuario General (Actor Principal):** Es el colaborador o empleado de la organización que labora en cualquiera de las sedes y puede iniciar sesión en el sistema cliente ingresando sus credenciales, redactar y enviar nuevos correos internos especificando el destinatario, asunto y cuerpo. Podrá consultar en su bandeja de salida para verificar los mensajes que ha emitido, seleccionar y leer el contenido completo de un mensaje que ya no requiera visualizar en sus bandejas.
- **Servicio de Autenticación FastAPI (Actor Secundario):** Es el componente externo basado en FastAPI encargado de centralizar la seguridad, la gestión de usuarios y la emisión/validación de credenciales.
  Puede validar credenciales de usuario enviadas desde el servidor de Go durante el inicio de sesión, emitir y retornar un token de sesión digital válido, comprobar la validez, vigencia y autenticidad del token de sesión en cada

transacción o petición de negocio que realice el servidor de Go, denegando el acceso de manera inmediata si el token es nulo, incorrecto o ha expirado.

3. ALCANCE

4.1. Operaciones que Realizará la Aplicación

El sistema cliente-servidor desarrollado en Go cubre la gestión y trazabilidad del flujo de mensajería interna entre las dos sedes de la organización:

- **Autenticación e inicio de sesión de usuario:** Conexión con el servicio externo en FastAPI para la validación de credenciales y obtención de token de sesión.
- **Redacción y envío de correos internos:** Creación y despacho de mensajes dirigidos a destinatarios válidos de la organización, registrando remitente, destinatario, asunto, cuerpo y marca de tiempo.
- **Consulta de bandeja de entrada:** Listado estructurado de los correos recibidos por el usuario autenticado, distinguiendo visual y lógicamente el estado (leído / no leído).
- **Consulta de bandeja de salida:** Listado de los correos emitidos por el usuario actual hacia otros colaboradores de cualquiera de las dos sedes.
- **Lectura de mensaje con actualización de estado:** Apertura del contenido completo de un correo específico y cambio automático de su estado a leído (leido = true).
- **Eliminación lógica de mensajes:** Borrado lógico (eliminado = true) de correos sin destruir la integridad referencial de los registros históricos en el repositorio.
- **Comunicación concurrente cliente-servidor:** Procesamiento concurrente de múltiples peticiones simultáneas desde diferentes sedes mediante *goroutines* y canales en Go.

4.2. Operaciones Adicionales del Sistema

- **Control de acceso y sesión activa por token:** Interceptación de cada petición de negocio para verificar la validez del token antes de procesar operaciones.
- **Aislamiento de persistencia en memoria segura:** Soporte de acceso concurrente seguro a las estructuras de almacenamiento mediante exclusión mutua (sync.RWMutex).

4.3. Exclusiones del Sistema

- **Integración con protocolos SMTP externos:** Se excluye el enlace con servidores de correo globales debido a que el sistema está diseñado

estrictamente para mensajería interna corporativa sobre la red privada interconectada.

- **Soporte y transferencia de archivos adjuntos pesados:** Excluido para prevenir congestión en los enlaces WAN/OSPF entre sedes y priorizar la entrega rápida y ligera de texto estructurado.
- **Recuperación o restablecimiento de contraseñas desde el cliente Go:** Excluido del alcance del cliente-servidor Go, delegando la gestión de cuentas y credenciales exclusivamente al servicio de autenticación en FastAPI.

4. REQUISITOS

5.1. Requisitos Funcionales y de Integración

Tabla 1

Matriz de requisitos funcionales y de integración del sistema cliente-servidor

CÓDIGO	REQUISITO	ACTOR	CRITERIO DE ACEPTACIÓN
RF-SW-01	Arquitectura Cliente-Servidor Desacoplada	Usuario General	El cliente ejecuta en un host de la Sede A y el servidor en un host de la Sede B; el cliente envía peticiones mediante sockets TCP/HTTP y recibe respuestas estructuradas en formato JSON sin compartir memoria ni binarios.
RF-SW-02.1	Autenticación e Inicio de Sesión	Usuario General / Servicio FastAPI	El consumidor envía usuario y contraseña; el sistema consulta al servicio FastAPI y, si las credenciales son correctas, retorna un token de sesión en formato string en menos de 2 segundos; si fallan, retorna error de credenciales inválidas y no genera sesión.
RF-SW-02.2	Redacción y Envío de Correo	Usuario General	El usuario ingresa destinatario, asunto y cuerpo; el sistema valida la existencia del destinatario, registra el correo con timestamp y retorna confirmación con código de éxito; si el

			destinatario no existe, retorna código de error y no almacena el correo.
RF-SW-02.3	Consulta de Bandeja de Entrada	Usuario General	El sistema retorna un listado de todos los correos dirigidos al usuario autenticado que no tengan borrado lógico, indicando claramente remitente, asunto, fecha y el indicador booleano de leído/no leído.
RF-SW-02.4	Consulta de Bandeja de Salida	Usuario General	El sistema retorna la totalidad de correos redactados y enviados por el usuario solicitante, mostrando destinatario, asunto, fecha y cuerpo del mensaje.
RF-SW-02.5	Lectura y Marcado de Mensaje	Usuario General	Al seleccionar un correo por su ID, el sistema muestra el cuerpo íntegro del mensaje y actualiza inmediatamente la propiedad leído a true en el repositorio; una consulta posterior debe reflejar dicho estado.
RF-SW-02.6	Eliminación Lógica de Mensaje	Usuario General	Al solicitar eliminar un correo por su ID, el sistema establece la opción *eliminado* en true; el correo deja de aparecer en las bandejas del usuario, pero el registro se mantiene para fines de auditoría.
RF-SW-03	Validación de Sesión en Operaciones Protegidas	Servicio FastAPI	Antes de procesar cualquier operación de correo, el servidor valida el token con FastAPI; si el token es nulo o inválido, deniega la acción de inmediato.
RF-INT-01	Operación Integral del Ecosistema	Usuario General /	El cliente Go enruta sus peticiones a través de la topología de red OSPF hacia el servidor Go, validando los

	Servicio FastAPI	tokens con el servicio FastAPI y persistiendo los datos de manera consistente sin pérdida de paquetes ni bloqueos en el flujo.

Nota. Requisitos adaptados a la arquitectura cliente-servidor en Go y al servicio de autenticación en FastAPI. Elaboración propia (2026).

5.2. Requisitos No Funcionales

Tabla 2

Especificación de requisitos no funcionales medibles

CÓDIGO	REQUISITO	ACTOR	VALOR MEDIBLE	CRITERIO DE ACEPTACIÓN
RNF-01	Rendimiento y Tiempo de Respuesta	Usuario General	Latencia ≤ 500 ms	El 95% de las operaciones locales de consulta y envío deben responder en un tiempo inferior o igual a 500 milisegundos bajo condiciones normales de red.
RNF-02	Concurrencia Simultánea	Servidor Go	Mínimo 20 clientes concurrentes	El servidor Go debe atender un mínimo de 20 conexiones simultáneas concurrentes utilizando *goroutines* sin presentar caídas, fugas de memoria ni bloqueos mutuos.

RNF-03	Disponibilidad Operativa	Servidor Go	Disponibilidad ≥ 99.0	El servidor de Go debe funcionar de manera continua durante todas las pruebas de envío y recepción entre las dos sedes, sin apagarse ni cerrarse por error al menos el 99% del tiempo.
RNF-04	Seguridad en Validación de Acceso	Servidor Go / Servicio FastAPI	100% de peticiones verificadas	El 100% de las solicitudes que lleguen al servidor de correo deben ser verificadas contra el validador de tokens antes de acceder a la capa de negocio.
RNF-05	Integridad y Concurrencia de Datos	Servidor Go	0 colisiones en memoria	Cuando varios usuarios lean o envíen correos a la vez, el sistema protege la información con un cerrojo (sync.RWMutex), logrando que la prueba oficial de Go (go test -race) termine con 0 advertencias o choques de datos.
RNF-06	Modularidad y Acoplamiento	Servidor Go	3 capas independientes compilables	El código se organiza en carpetas separadas e

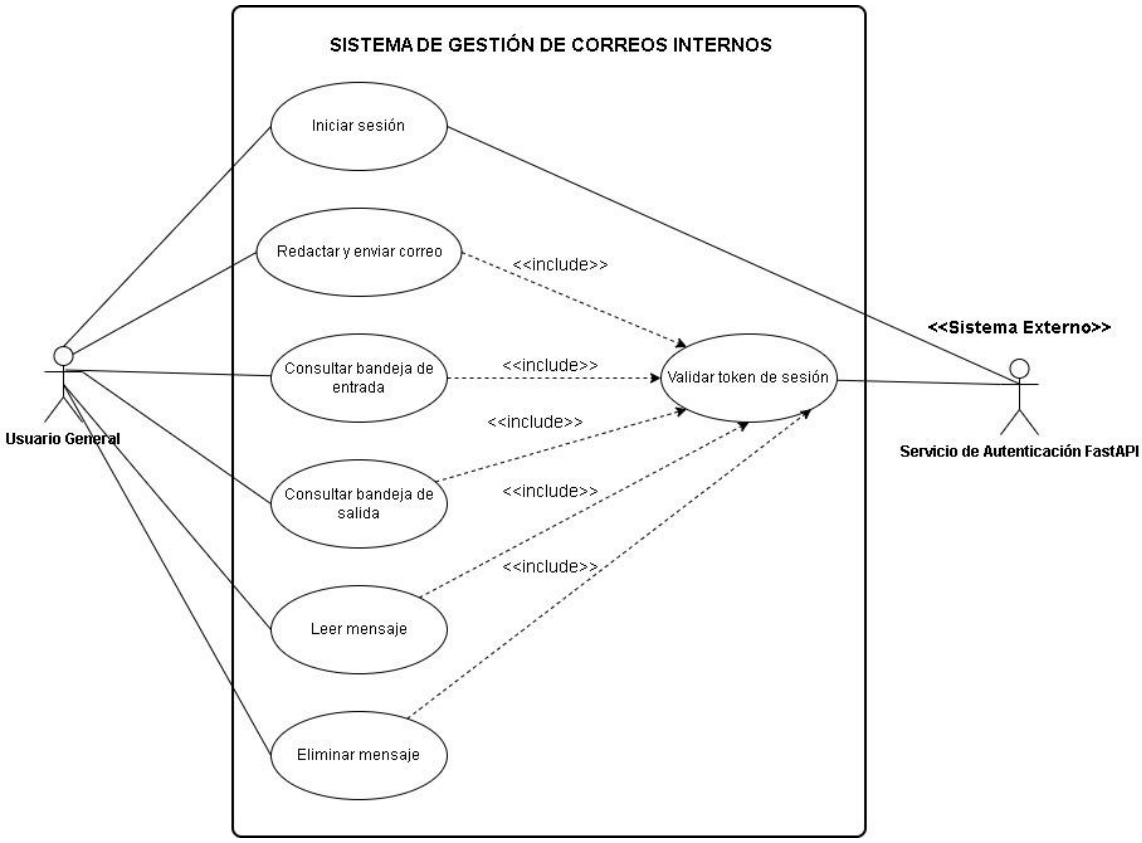
				independientes (red, lógica y datos) que no dependen unas de otras en círculo, permitiendo que cada carpeta compile de forma individual sin errores.
RNF-07	Tolerancia a Fallos y Manejo de Errores	Servidor Go / Servicio FastAPI	Tasa de caídas forzosas = 0	Si un usuario ingresa datos erróneos, busca un correo inexistente o falla la conexión con FastAPI, el servidor atrapa el fallo y responde con un mensaje de error sin apagarse ni cerrarse bruscamente.

Nota. Métricas cuantitativas definidas para la evaluación de desempeño y estabilidad del software. Elaboración propia.

5. DIAGRAMA DE CASOS DE USO

Figura 1

Diagrama de casos de uso del Sistema de Gestión de Correos Internos en Go



Nota. Representación en notación UML que muestra de manera gráfica las interacciones del Usuario General, las inclusiones obligatorias para la verificación de tokens de sesión y la delegación al Servicio de Autenticación externo en FastAPI. Elaboración propia (2026).

6. FICHAS DE CASOS DE USO

Tabla 3

Ficha del caso de uso CU-01: Iniciar sesión

Elemento	Descripción

Actores	Usuario General (principal), Servicio FastAPI (secundario).
Requisitos	RF-SW-02.1, RF-SW-01, RF-INT-01, RNF-07.
Precondición	El cliente tiene conexión con el servidor y el usuario existe en FastAPI.
Flujo principal	1. El Usuario General ingresa usuario y contraseña en el cliente. 2. El cliente envía la solicitud de inicio de sesión al servidor Go. 3. El servidor reenvía las credenciales a FastAPI. 4. FastAPI las valida y devuelve un token de sesión (en menos de 2 segundos). 5. El servidor responde al cliente con el token. 6. El cliente guarda el token y el usuario activo, y habilita las operaciones de correo.
Excepciones	• **Contraseña o usuario incorrectos:** FastAPI no emite token, el cliente muestra “credenciales inválidas” y no se crea sesión. • **Campos vacíos:** el cliente no envía la petición y pide completar los datos. • **FastAPI no responde:** el servidor agota el tiempo de espera y responde “servicio de autenticación no disponible”, sin caerse. • **Servidor Go no responde:** el cliente informa el fallo de conexión y permite reintentar.

Nota. Elaboración propia (2026).

Tabla 4

Ficha del caso de uso CU-02: Redactar y enviar correo

Elemento	**Descripción**
Actores	Usuario General (principal), Servicio FastAPI (secundario).

Requisitos	RF-SW-02.2, RF-SW-03, RF-INT-01, RNF-04, RNF-05.
Precondición	Sesión iniciada con token vigente.
Flujo principal	1. El Usuario General ingresa destinatario, asunto y cuerpo del mensaje. 2. El cliente envía la petición al servidor junto con el token. 3. El servidor valida el token con el Servicio FastAPI. 4. El servicio de correo consulta a FastAPI si el destinatario existe. 5. El servicio crea el correo con el remitente tomado del token, la fecha y hora de envío, leído = false y eliminado = false. 6. El repositorio guarda el correo. 7. El servidor responde con código de éxito y el cliente confirma el envío.
Excepciones	• **Destinatario inexistente:** se retorna un código de error y no se almacena nada. • **Asunto o cuerpo vacíos:** el servidor rechaza la petición con un mensaje de error. • **Token nulo, inválido o vencido:** se deniega la operación y se pide iniciar sesión. • **FastAPI no responde:** se deniega la operación con el mensaje “servicio de autenticación no disponible”, no se guarda nada y el servidor sigue funcionando. • **Servidor Go no responde:** el cliente informa que el correo no se envió.

Nota. Elaboración propia (2026).

Tabla 5

Ficha del caso de uso CU-03: Consultar bandeja de entrada

Elemento	Descripción
Actores	Usuario General (principal), Servicio FastAPI (secundario).
Requisitos	RF-SW-02.3, RF-SW-03, RF-INT-01, RNF-01.
Precondición	Sesión iniciada con token vigente.
Flujo principal	1. El Usuario General selecciona la opción “Bandeja de entrada”. 2. El cliente envía la petición al servidor junto con el token. 3. El servidor valida el token con el Servicio FastAPI.

	4. El servicio solicita al repositorio los correos dirigidos al usuario con eliminado = false. 5. El cliente muestra la lista con remitente, asunto, fecha e indicador de leído / no leído.
Excepciones	• **Bandeja vacía:** se muestra una lista vacía con el aviso “No hay mensajes”; no se considera un error. • **Token nulo, inválido o vencido:** se deniega la operación y se exige iniciar sesión. • **FastAPI no responde:** se deniega la operación con “servicio de autenticación no disponible” y el servidor sigue funcionando. • **Servidor Go no responde:** el cliente informa el fallo de conexión y permite reintentar.

Nota. Elaboración propia (2026).

Tabla 6

Ficha del caso de uso CU-04: Consultar bandeja de salida

Elemento	Descripción
Actores	Usuario General (principal), Servicio FastAPI (secundario).
Requisitos	RF-SW-02.4, RF-SW-03, RF-INT-01, RNF-01.
Precondición	Sesión iniciada con token vigente.
Flujo principal	1. El Usuario General selecciona la opción “Bandeja de salida”. 2. El cliente envía la petición al servidor junto con el token. 3. El servidor ejecuta CU-07 (Validar token de sesión). 4. El servicio solicita al repositorio los correos enviados por el usuario con eliminado = false. 5. El cliente muestra la lista con destinatario, asunto, fecha y cuerpo del mensaje.
Excepciones	• **Bandeja vacía:** se muestra una lista vacía con el aviso “No hay mensajes”; no se considera un error. • **Token nulo, inválido o vencido:** se deniega la operación y se exige iniciar sesión. • **FastAPI no responde:** se deniega la operación con “servicio de autenticación no disponible” y el servidor sigue funcionando.

	• **Servidor Go no responde:** el cliente informa el fallo de conexión y permite reintentar.

Nota. Elaboración propia (2026).

Tabla 7

Ficha del caso de uso CU-05: Leer mensaje

Elemento	Descripción
Actores	Usuario General (principal), Servicio FastAPI (secundario).
Requisitos	RF-SW-02.5, RF-SW-03, RF-INT-01, RNF-05.
Precondición	Sesión iniciada con token vigente y mensaje visible en alguna bandeja.
Flujo principal	1. El Usuario General selecciona un mensaje de la lista. 2. El cliente envía el ID del mensaje al servidor junto con el token. 3. El servidor ejecuta CU-07 (Validar token de sesión). 4. El servicio busca el correo por su ID y comprueba que el usuario sea su remitente o su destinatario. 5. Si el usuario es el destinatario, el repositorio actualiza el correo a leído = true. 6. El cliente muestra el cuerpo completo del mensaje; una consulta posterior refleja el nuevo estado.
Excepciones	• **ID inexistente o correo ya eliminado:** se retorna “mensaje no encontrado” y no se modifica nada. • **El correo pertenece a otro usuario:** se deniega el acceso. • **Token nulo, inválido o vencido:** se deniega la operación y se exige iniciar sesión. • **FastAPI no responde:** se deniega la operación, el estado del correo no cambia y el servidor sigue funcionando. • **Servidor Go no responde:** el cliente informa el fallo y el estado del correo no cambia.

Nota. Elaboración propia (2026).

Tabla 8

Ficha del caso de uso CU-06: Eliminar mensaje

Elemento	Descripción

Actores	Usuario General (principal), Servicio FastAPI (secundario).
Requisitos	RF-SW-02.6, RF-SW-03, RF-INT-01, RNF-05.
Precondición	Sesión iniciada con token vigente y mensaje visible en alguna bandeja.
Flujo principal	1. El Usuario General selecciona un mensaje y elige la opción “Eliminar”. 2. El cliente envía el ID del mensaje al servidor junto con el token. 3. El servidor ejecuta CU-07 (Validar token de sesión). 4. El servicio comprueba que el correo pertenezca al usuario. 5. El repositorio aplica el borrado lógico (eliminado = true) y conserva el registro. 6. El cliente confirma la acción y el mensaje deja de aparecer en las bandejas.
Excepciones	• **ID inexistente o correo ya eliminado:** se retorna “mensaje no encontrado” y no se modifica nada. • **El correo pertenece a otro usuario:** se deniega la operación. • **Token nulo, inválido o vencido:** se deniega la operación y se exige iniciar sesión. • **FastAPI no responde:** se deniega la operación, el correo sigue visible y el servidor sigue funcionando. • **Servidor Go no responde:** el cliente informa el fallo y el correo sigue visible.

Nota. Elaboración propia (2026).

Tabla 9

Ficha del caso de uso CU-07: Validar token de sesión

Elemento	Descripción
Actores	Sin actor propio: se incluye (<<include>>) en CU-02 a CU-06. Servicio FastAPI (secundario).
Requisitos	RF-SW-03, RNF-04, RNF-07.
Precondición	Llega al servidor una petición de negocio con su token.
Flujo principal	1. El servidor recibe la petición junto con el token. 2. Antes de ejecutar la operación, el servicio de correo solicita la validación del token.

	3. El cliente FastAPI del servidor consulta al servicio de autenticación. 4. FastAPI confirma que el token es válido y devuelve el usuario asociado. 5. El servicio continúa la operación con el usuario identificado.
Excepciones	• **Token nulo o con formato inválido:** se deniega de inmediato, sin consultar a FastAPI. • **Token vencido o rechazado por FastAPI:** se deniega la operación y se pide iniciar sesión. • **FastAPI no responde:** se deniega la operación (ante la duda no se da acceso) y el servidor sigue funcionando.

Nota. Elaboración propia (2026).

7. MATRIZ DE TRAZABILIDAD

Tabla 10

Matriz de trazabilidad de requisitos, casos de uso, clases y pruebas

Requisito	Caso de uso	Clase y método	Prueba
RF-SW-01	CU-01 a CU-06	ClienteCorreo.Conectar() ServidorCorreo.Iniciar() ServidorCorreo.atenderConexion()	Ejecutar el cliente en un equipo de la Sede A y el servidor en uno de la Sede B; enviar un correo y verificar que la respuesta llegue en formato JSON.
RF-SW-02.1 a 02.6	CU-01 a CU-06	ClienteCorreo.IniciarSesion() ServicioCorreo.IniciarSesion() (nuevo) ValidadorAuth.Autenticar() (nuevo) ClienteFastAPI.Autenticar() (nuevo)	Iniciar sesión con credenciales correctas: se recibe un

			token en menos de 2 s. Con credenciales incorrectas: error y sin token.
RF-SW-03	CU-02 a CU-06	ValidadorAuth.ValidarToken(), ClienteFastAPI.ValidarToken(), ServicioCorreo.EnviarCorreo() y demás operaciones de ServicioCorreo	Ejecutar cada operación sin token, con token alterado y con token vencido: todas son denegadas y no cambia ningún dato.
RF-INT-01	CU-01 a CU-06	ClienteCorreo.Conectar(), ServidorCorreo.atenderConexion(), ServicioCorreo.EnviarCorreo(), ClienteFastAPI.ValidarToken(), RepositorioMemoria.Guardar()	Prueba integral entre sedes con FastAPI real: iniciar sesión, enviar, consultar ambas bandejas, leer y eliminar, sin errores ni pérdida de datos.
RNF-01	CU-02 a CU-06	ServicioCorreo.EnviarCorreo(), ServicioCorreo.ConsultarBandejaEntrada(), RepositorioMemoria.ListarBandejaEntrada()	Medir 100 operaciones de consulta y envío: el percentil 95 debe ser ≤ 500 ms.

RNF-02	CU-01 a CU-06	ServidorCorreo.atenderConexion(), ServidorCorreo.procesarTrabajadorConcurrente()	Lanzar 20 clientes simultáneos con operaciones mixtas: sin caídas, fugas de memoria ni bloqueos.
RNF-03	CU-01 a CU-06	ServidorCorreo.Iniciar()	Mantener el servidor encendido durante toda la sesión de pruebas entre sedes y registrar el tiempo activo (mínimo 99 %).
RNF-04	CU-02 a CU-06	ValidadorAuth.ValidarToken()	Enviar peticiones con token nulo, inválido y válido a cada operación: el 100 % pasa por el validador antes de la capa de negocio.
RNF-05	CU-02 a CU-06	RepositorioMemoria.Guardar(), ListarBandejaEntrada(), ActualizarEstadoLeido(), EliminarLogico() (mu sync.RWMutex)	Ejecutar go test -race con lecturas, envíos y eliminaciones concurrentes:

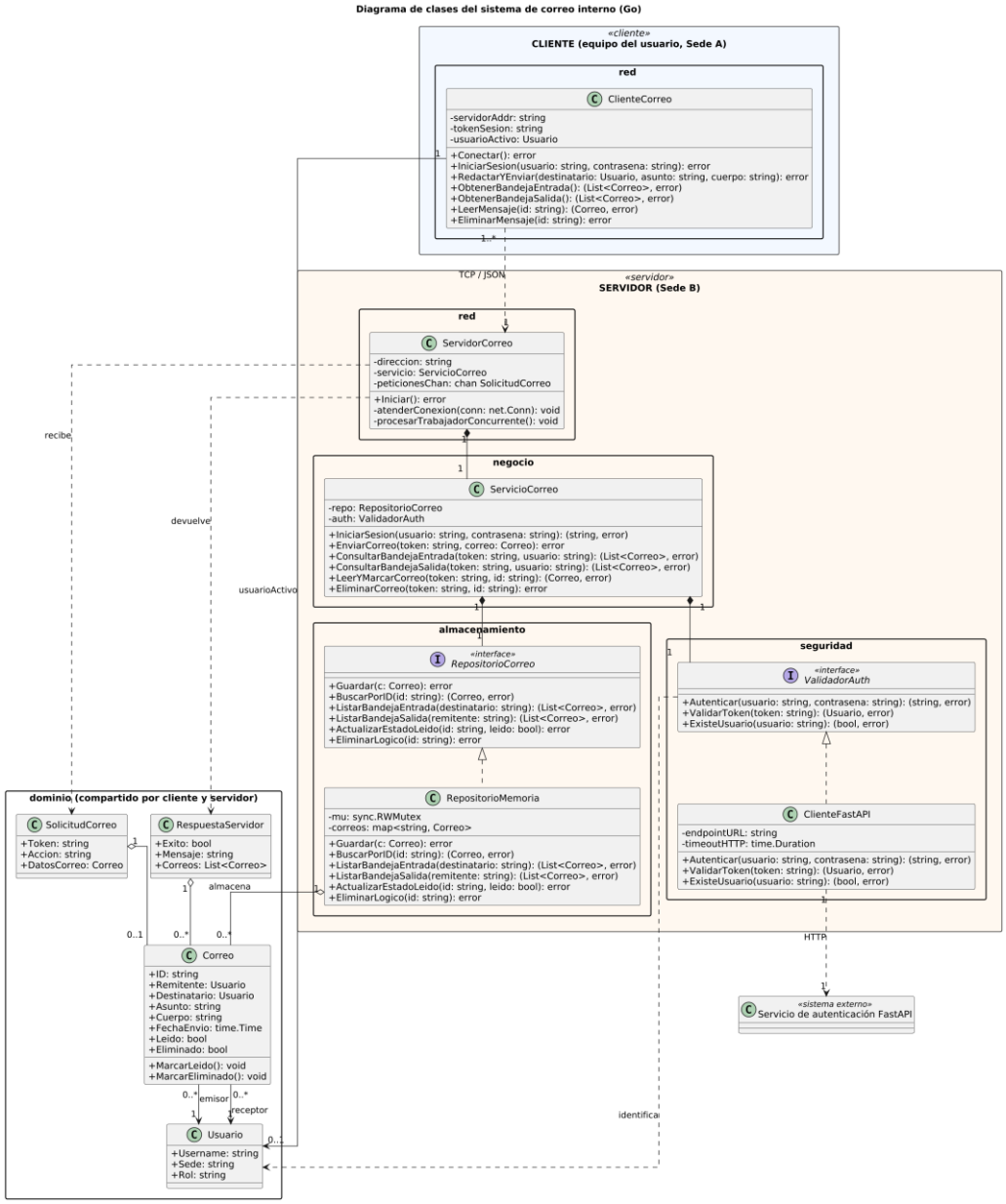
			0 advertencias.
RNF-06	CU-01 a CU-06	ServidorCorreo.Iniciar() (red), ServicioCorreo.EnviarCorreo() (negocio), RepositorioMemoria.Guardar() (almacenamiento)	Compilar cada paquete por separado (go build) y comprobar que no existan dependencias circulares.
RNF-07	CU-01 a CU-06	ServidorCorreo.atenderConexion(), ServicioCorreo.LeerYMarcarCorreo(), ClienteFastAPI.ValidarToken(), RespuestaServidor	Enviar datos inválidos, pedir un ID inexistente y apagar FastAPI: el servidor responde con un error y sigue ejecutándose.

Nota. Los métodos marcados con *nuevo* se agregan al diagrama de clases corregido. Elaboración propia (2026).

Para que cada requisito pueda atenderse con una clase y un método concretos, se corrigió el diagrama de clases de la Fase 1.A continuación se presenta el diagrama actualizado, en el que se separan la parte cliente y la parte servidor, se indican las multiplicidades de cada relación y se incorporan los métodos que faltaban (Autenticar, ExisteUsuario e IniciarSesion), marcados como nuevos en la Tabla 10.

Figura 2

Diagrama de clases corregido del sistema de correo interno en Go



Nota. El diagrama separa la parte cliente (equipo del usuario) de la parte servidor y muestra las multiplicidades de cada relación. El servicio de autenticación FastAPI se representa únicamente como sistema externo, accedido desde el servidor mediante la clase ClienteFastAPI. Elaboración propia (2026).

8. CAMBIOS RESPECTO A LA FASE 1

Tras la entrega de la Fase 1, el docente formuló observaciones sobre la ubicación de los documentos en el repositorio y sobre el diagrama de clases. La Tabla 11 resume cada observación y la corrección realizada.

Tabla 11

Observaciones recibidas en la Fase 1 y correcciones realizadas

Observación recibida	Corrección realizada	Evidencia
En el diagrama conviene poner cuántos participan en cada relación.	Se agregaron multiplicidades a todas las relaciones del diagrama de clases; por ejemplo, 1..* ClienteCorreo se conecta con 1 ServidorCorreo, y un Correo tiene 1 emisor y 1 receptor.	Sección 8, Figura 2
En el diagrama conviene separar con recuadros la parte cliente y la parte servidor.	El diagrama muestra un recuadro para el cliente y otro para el servidor; el paquete de dominio, compartido por ambos, y el servicio externo FastAPI se representan fuera de los dos recuadros.	Sección 8, Figura 2

Nota. Elaboración propia (2026).

Además, al pasar del análisis a la especificación formal del sistema, el equipo ajustó otros elementos de la Fase 1. La Tabla 12 compara su estado en ambas fases.

Tabla 12

Ajustes adicionales realizados por el equipo en la Fase 2

Aspecto	Fase 1	Fase 2
Requisitos funcionales	RF-SW-01 a RF-SW-06, con RF-SW-02 como un solo requisito.	RF-SW-02 se dividió en una fila por operación (RF-SW-02.1 a RF-SW-02.6) y se añadió RF-INT-01 para la operación integral del sistema.
Requisitos no funcionales	RF-SW-04 (modularidad), RF-SW-05 (errores) y RF-SW-06 (concurrencia), sin valores medibles.	Pasaron a RNF-06, RNF-07 y RNF-02, y se agregaron RNF-01 a RNF-07, cada uno con un valor y un criterio de aceptación medibles.

Actores	Usuario General, Administrador de Red y Sistemas, y Directiva Institucional.	Usuario General y Servicio de Autenticación FastAPI. Los roles de administración y directiva no usan la aplicación, por lo que salieron del modelo.
Casos de uso	No incluidos.	Diagrama de casos de uso y siete fichas (CU-01 a CU-07), con flujo principal y excepciones.
Trazabilidad	No incluida.	Matriz que une cada requisito con su caso de uso, clase, método y prueba.
Diagrama de clases	Sin métodos de autenticación de credenciales ni de verificación de destinatario.	Se agregaron Autenticar y ExisteUsuario (ValidadorAuth y ClienteFastAPI) e IniciarSesion (ServicioCorreo), y IniciarSesion de ClienteCorreo pasó a recibir usuario y contraseña.

Nota. Elaboración propia (2026).

9. DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL

Durante el desarrollo de esta fase, el equipo utilizó la inteligencia artificial Gemini (Google) como apoyo para revisar conceptos técnicos, corrección de errores en el diagrama UML actualizado y asistir en correcciones de redacción del documento. La herramienta sirvió como guía de consulta para asegurar que la entrega cumpla con lo solicitado por la docente.

Prompts Utilizados

- *"Revisa mi diagrama UML en base a esta retroalimentación y dime si cumple con los cambios necesarios."*
- *"Explícame de forma clara cómo funciona sync.RWMutex en Go cuando varios usuarios leen o envían correos al mismo tiempo."*

Criterio Propio Aplicado

- El diagrama fue modificado por los integrantes del equipo en Draw.io, revisando que cada flecha <<include>> tuviera sentido real dentro del flujo del sistema, corrigiendo las conexiones erróneas y añadiendo las multiplicidades de cada relación pedidas en la retroalimentación.

- La IA sugería incluir administradores o supervisores de red, pero el equipo decidió dejar únicamente a Usuario General y al servicio de FastAPI, ya que las configuraciones de red pertenecen a otra asignatura y no al código de esta materia.
- Adaptamos las explicaciones teóricas de la IA para que coincidan exactamente con las herramientas que usamos en Go, como la prueba de colisiones go test -race y el manejo ordenado de errores con error, asegurando que todo lo descrito sea comprobable en nuestro programa.